

Multi-Region Domain Deployment Guide

Complete guide for deploying kidrove.in and kidrove.ae alongside existing kidrove.com.

Prerequisites Checklist

- [] DNS records configured for all 9 domains (see below)
- [] DNS propagation verified (1-24 hours)
- [] Server access: `ssh root@93.127.185.245`
- [] Backup of current nginx configuration created

Part 1: DNS Configuration

DNS Records to Create

For **kidrove.in**:

Type: A	Name: @	Value: 93.127.185.245
Type: A	Name: www	Value: 93.127.185.245
Type: A	Name: api	Value: 93.127.185.245

For **kidrove.ae**:

Type: A	Name: @	Value: 93.127.185.245
Type: A	Name: www	Value: 93.127.185.245
Type: A	Name: api	Value: 93.127.185.245

Verify DNS Propagation

```
# Check each domain resolves to server IP
nslookup kidrove.in
nslookup www.kidrove.in
nslookup api.kidrove.in
nslookup kidrove.ae
nslookup www.kidrove.ae
nslookup api.kidrove.ae

# Expected output for each: 93.127.185.245
```

Important: Wait for DNS propagation before proceeding (1-24 hours typically).

Part 2: Backend CORS Configuration

Update backend/.env.production on Server

SSH into server:

```
ssh root@93.127.185.245
cd /var/www/GEMA-Project/backend
```

Edit `.env.production`:

```
nano .env.production
```

Find and update these lines:

```
# OLD:
-FRONTEND_URL=https://your-domain.com
-ADDITIONAL_ALLOWED_ORIGINS=

# NEW:
+FRONTEND_URL=https://kidrove.com
+ADDITIONAL_ALLOWED_ORIGINS=https://www.kidrove.com,https://kidrove.in,https://www.kidrove.in,https://kidrove.ae,https://www.kidrove.ae
```

Save and exit (Ctrl+O, Enter, Ctrl+X in nano).

Explanation:

- `FRONTEND_URL`: Primary domain for CORS
 - `ADDITIONAL_ALLOWED_ORIGINS`: Comma-separated list of additional allowed origins
 - Backend CORS config (server.ts:41-95) already supports this format
-

Part 3: SSL Certificate Acquisition

Run Certbot on Server

```
# 1. Verify existing kidrove.com certs
certbot certificates

# 2. Request api.kidrove.com (if not already exists)
certbot certonly --nginx -d api.kidrove.com

# 3. Request kidrove.in + www (SAN cert)
certbot certonly --nginx -d kidrove.in -d www.kidrove.in

# 4. Request api.kidrove.in
certbot certonly --nginx -d api.kidrove.in

# 5. Request kidrove.ae + www (SAN cert)
certbot certonly --nginx -d kidrove.ae -d www.kidrove.ae

# 6. Request api.kidrove.ae
certbot certonly --nginx -d api.kidrove.ae

# 7. Verify all 6 certificates obtained
certbot certificates
```

Expected certificates:

1. kidrove.com (with www.kidrove.com as SAN)
2. api.kidrove.com
3. kidrove.in (with www.kidrove.in as SAN)
4. api.kidrove.in
5. kidrove.ae (with www.kidrove.ae as SAN)
6. api.kidrove.ae

Auto-renewal: Already configured via systemd timer (runs twice daily).

Part 4: Nginx Configuration Deployment

Step 1: Backup Current Configuration

```
cd /etc/nginx/sites-available
cp nginx-subdomain.conf nginx-subdomain.conf.backup-$(date +%F)
```

Step 2: Upload New Files from Local Machine

From your local development machine (Windows):

```
# Upload nginx config
scp E:\coding\gema\nginx-multi-region.conf root@93.127.185.245:/etc/nginx/sites-available/

# Upload snippet files
scp E:\coding\gema\deployment\nginx\snippets\*.conf root@93.127.185.245:/etc/nginx/snippets/
```

Alternative (if using Git on server):

```
cd /var/www/GEMA-Project
git pull
cp nginx-multi-region.conf /etc/nginx/sites-available/
cp deployment/nginx/snippets/*.conf /etc/nginx/snippets/
```

Step 3: Enable New Configuration

```
cd /etc/nginx/sites-enabled
rm -f * # Remove old symlinks
ln -s ../sites-available/nginx-multi-region.conf .
```

Step 4: Test and Reload Nginx

```
# Test configuration syntax
nginx -t

# Expected output:
# nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
# nginx: configuration file /etc/nginx/nginx.conf test is successful
```

If test passes:

```
# Reload nginx (zero-downtime)
systemctl reload nginx
```

If test fails:

```
# Check error details
nginx -t

# Rollback if needed (see Rollback section)
```

Part 5: Backend Deployment

Reload PM2 with New CORS Settings

```
cd /var/www/GEMA-Project/backend

# Verify .env.production has new CORS settings (from Part 2)
grep FRONTEND_URL .env.production
grep ADDITIONAL_ALLOWED_ORIGINS .env.production

# Reload PM2 (zero-downtime)
pm2 reload ecosystem.config.js --env production

# Check backend status
pm2 status
pm2 logs gema-backend --lines 50 | grep -i cors
```

Verify Backend Health

```
# Test health endpoint
curl http://localhost:5000/health

# Expected: {"status":"ok",...}
```

Part 6: Frontend Deployment

Deploy Updated Frontend with Runtime API Detection

```
cd /var/www/GEMA-Project/frontend

# Pull latest code with frontend/src/config/api.ts changes
git pull

# Install dependencies
npm install

# Build production frontend
npm run build

# Reload nginx to serve new build
systemctl reload nginx
```

Part 7: Testing & Verification

Health Check Matrix

Test all 9 domains in browser or with curl:

```
# Frontend domains (should load React app)
curl -I https://kidrove.com
curl -I https://www.kidrove.com
curl -I https://kidrove.in
curl -I https://www.kidrove.in
curl -I https://kidrove.ae
curl -I https://www.kidrove.ae

# API domains (should return JSON)
curl https://api.kidrove.com/health
curl https://api.kidrove.in/health
curl https://api.kidrove.ae/health
```

Expected Results:

Domain	Expected Response
https://kidrove.com	HTTP 200, HTML (React app)
https://www.kidrove.com	HTTP 200, HTML (React app)
https://api.kidrove.com/health	{"status": "ok"}
https://kidrove.in	HTTP 200, HTML (React app)
https://www.kidrove.in	HTTP 200, HTML (React app)
https://api.kidrove.in/health	{"status": "ok"}
https://kidrove.ae	HTTP 200, HTML (React app)
https://www.kidrove.ae	HTTP 200, HTML (React app)
https://api.kidrove.ae/health	{"status": "ok"}

CORS Testing

From browser console on each domain:

```
// Test from kidrove.in
fetch('https://api.kidrove.in/api/health')
  .then(r => r.json())
  .then(console.log);
// Should NOT show CORS errors

// Test from kidrove.ae
fetch('https://api.kidrove.ae/api/health')
  .then(r => r.json())
  .then(console.log);
// Should NOT show CORS errors
```

SSL Certificate Verification

```
# Check certificate details for each domain
echo | openssl s_client -connect kidrove.in:443 -servername kidrove.in 2>/dev/null | openssl x509 -noout -dates
echo | openssl s_client -connect api.kidrove.in:443 -servername api.kidrove.in 2>/dev/null | openssl x509 -noout -dates
echo | openssl s_client -connect kidrove.ae:443 -servername kidrove.ae 2>/dev/null | openssl x509 -noout -dates
echo | openssl s_client -connect api.kidrove.ae:443 -servername api.kidrove.ae 2>/dev/null | openssl x509 -noout -dates
```

Functional Testing Checklist

- ☐ User registration from kidrove.in works
- ☐ User login from kidrove.ae works
- ☐ API calls work from all domains
- ☐ File uploads work (Cloudinary)
- ☐ Payment flows work (Stripe)
- ☐ Email sending works

- [] No CORS errors in browser console

Part 8: Monitoring

Log Monitoring

```
# Watch nginx logs
tail -f /var/log/nginx/kidrove-in-access.log
tail -f /var/log/nginx/kidrove-ae-access.log
tail -f /var/log/nginx/api-kidrove-in-error.log
tail -f /var/log/nginx/api-kidrove-ae-error.log

# PM2 logs (check for CORS errors)
pm2 logs gema-backend --lines 100 | grep -i "cors\|origin"

# SSL certificate expiry check
certbot certificates
```

SSL Certificate Monitoring

Expected: 6 certificates, all with 60+ days validity.

Auto-renewal runs twice daily via systemd timer:

```
# Check renewal timer status
systemctl status certbot.timer

# Test renewal (dry-run)
certbot renew --dry-run
```

Rollback Procedure

If Issues Arise

Immediate Rollback (< 5 minutes):

```
# On server
cd /etc/nginx/sites-enabled
rm -f nginx-multi-region.conf
ln -s ../sites-available/nginx-subdomain.conf.backup-YYYY-MM-DD .
nginx -t && systemctl reload nginx

# Revert backend CORS
cd /var/www/GEMA-Project/backend
# Restore old .env.production from backup
cp .env.production.backup .env.production
pm2 reload ecosystem.config.js --env production
```

Partial Rollback (disable new domains, keep .com):

Edit nginx-multi-region.conf and comment out kidrove.in and kidrove.ae server blocks:

```
nano /etc/nginx/sites-available/nginx-multi-region.conf
# Comment out lines for .in and .ae (add # at start of each line)
nginx -t && systemctl reload nginx
```

Summary of Changes

Files Created/Modified on Local Machine:

1. ☒ frontend/src/config/api.ts - Runtime API endpoint detection
2. ☒ frontend/src/services/api.ts - Updated to use runtime detection
3. ☒ frontend/src/config/app.ts - Updated to use runtime detection
4. ☒ frontend/src/config.ts - Updated to use runtime detection
5. ☒ frontend/src/pages/auth/LoginPage.tsx - Updated debug logging
6. ☒ frontend/src/contexts/CurrencyContext.tsx - Updated to use runtime detection
7. ☒ deployment/nginx/snippets/ssl-params.conf - SSL configuration snippet

8. ☒ deployment/nginx/snippets/security-headers.conf - Security headers snippet
9. ☒ deployment/nginx/snippets/api-proxy.conf - API proxy configuration snippet
10. ☒ nginx-multi-region.conf - Complete multi-region nginx config (12 server blocks)

Files Modified on Server:

1. /var/www/GEMA-Project/backend/.env.production - Updated CORS configuration
2. /etc/nginx/sites-available/nginx-multi-region.conf - New nginx config
3. /etc/nginx/snippets/*.conf - New snippet files
4. /etc/nginx/sites-enabled/ - Symlink updated

No Changes Needed:

- backend/ecosystem.config.js - Port 5000 already correct
 - backend/src/server.ts - CORS logic already supports comma-separated origins
-

Next Steps

After successful deployment:

1. **Monitor logs** for 24-48 hours for any CORS errors or issues
 2. **Update deploy.sh** to include health checks for all 3 API domains
 3. **Update documentation** with new domains
 4. **Update Google Analytics** (if used) to track all 3 domains
 5. **Update sitemap.xml** for each domain
 6. **Consider region-specific features** (currency, language) for future phases
-

Support & Troubleshooting

Common Issues

Issue: CORS errors after deployment

- Solution: Check backend/.env.production has all domains in ADDITIONAL_ALLOWED_ORIGINS
- Debug: pm2 logs gema-backend | grep -i cors

Issue: SSL certificate acquisition fails

- Solution: Ensure DNS has propagated (wait 24h if needed)
- Debug: nslookup domain-name

Issue: Wrong API endpoint called from frontend

- Solution: Check browser console for API_BASE_URL value
- Debug: Add console.log('Using API:', API_BASE_URL) in frontend

Issue: Nginx config test fails

- Solution: Check syntax errors in config file
 - Debug: nginx -t shows exact error location
-

Contact

For deployment issues:

- Server Logs: /var/log/nginx/ and pm2 logs
- Configuration: E:\coding\gema\ (local) or /var/www/GEMA-Project/ (server)